IF2224 - Teori Bahasa Formal dan Otomata

Tugas Besar

Milestone 1 : Lexical Analysis

12 Maret 2026 - 2 April 2026

Mirza Tsabita Wafa'ana	13524114
Nathanael Shane Bennet	13524119
Faris Wirakusuma Triawan	13524130
Jonathan Harijadi	13524144

Program Studi Teknik Informatika

Sekolah Teknik Elektro dan Informatika

Institut Teknologi Bandung

2026

DAFTAR ISI

BAB 1 : Teori Singkat

1.1. Kompilator (*Compiler*)

Kompilator merupakan suatu program yang bertugas menerjemahkan kode sumber (*source code*) yang ditulis dalam bahasa pemrograman tingkat tinggi menjadi kode yang dapat dieksekusi oleh mesin. Proses kompilasi secara umum terbagi menjadi beberapa tahap, yaitu analisis leksikal (*lexical analysis*), analisis sintaksis (*syntax analysis*), analisis semantik (*semantic analysis*), optimasi kode, dan pembangkitan kode target. Tahap pertama dan paling mendasar dari proses kompilasi adalah analisis leksikal, yang menjadi fokus utama pada proyek ini (Aho, Lam, Sethi, & Ullman, 2006).

1.2. Analisis Leksikal (Lexical Analyzer / Lexer)

Analisis leksikal adalah tahap pertama dalam proses kompilasi. Fungsi utamanya adalah membaca baris karakter dari kode sumber dan mengelompokkannya menjadi unit-unit bermakna yang disebut *lexeme*. Lexer juga bertugas membuang karakter yang tidak relevan seperti *whitespace*, tab, dan komentar.

Dalam implementasi ini, *Lexer* menggunakan prinsip Longest Match Strategy, di mana mesin akan mencoba menyerap karakter sebanyak mungkin selama urutan tersebut masih membentuk pola yang valid (Aho et al., 2006).

1.3. Token dan Lexeme

Terdapat perbedaan teknis antara *Token*, *Lexeme*, dan *Pattern*:

- Lexeme: Rangkaian karakter aktual dalam kode sumber yang cocok dengan pola tertentu (misal: 123, temp, :=).
- Token: Pasangan yang terdiri dari nama token (tipe) dan nilai atribut opsional. Nama token adalah simbol abstrak yang mewakili unit leksikal (misal: intcon, ident, becomes).
- Pattern: Deskripsi aturan yang harus dipenuhi oleh rangkaian karakter agar dapat menjadi sebuah token. Dalam proyek ini, pola didefinisikan menggunakan logika DFA.

1.4. Deterministic Finite Automaton (DFA)

DFA adalah model matematika yang digunakan untuk mengenali pola bahasa reguler. Secara formal, DFA didefinisikan sebagai 5-tuple M = (Q, Σ, δ, q_0, F) di mana:

- Q: Himpunan dari *states* (keadaan).
- Σ: Alfabet input (karakter ASCII).
- δ: Fungsi transisi δ : Q x E -> Q
- q_0: *Start state* (keadaan awal).
- F: Himpunan *accepting states* (keadaan akhir).

1.5. Tabel Transisi State

Tabel transisi adalah representasi tabular dari fungsi δ. Tabel ini memetakan hubungan antara *current state* dan karakter input untuk menentukan *next state*. Dalam pemrograman C++, tabel ini diwujudkan melalui struktur kontrol switch-case dan if-else yang efisien, yang secara fungsional identik dengan pencarian pada matriks transisi.

1.6. Data Flow Diagram (DFD)

DFD digunakan untuk menggambarkan aliran data dalam sistem *Lexical Analyzer*.

- Entitas Luar: Pengguna atau file teks yang menyediakan kode sumber.
- Proses: Transformasi teks mentah melalui pemindaian karakter, pembangunan *buffer*, hingga pemetaan token.
- Data Store: Penyimpanan internal seperti keywordMap yang digunakan untuk validasi *lexeme* terhadap kata kunci yang telah dipesan (*reserved words*).
- Output: Aliran token yang siap dikonsumsi oleh tahap *Syntax Analysis* (Parser).

BAB 2 : Perancangan & Implementasi

2.2 Implementasi Program

Program dibuat menggunakan bahasa C++ dan implementasi lexer dan dfa terdapat pada file file berikut: lexer.cpp dan lexer.hpp, token.hpp dan dfa_graph.hpp.

1. File token.hpp

File ini bertugas sebagai definisi struktur data fundamental yang akan menjadi *output* dari *lexer* dan *input* bagi *parser*. Didalam file ini terdapat enum yang berisikan token-token yang nantinya akan dibaca oleh lexical analyzer. Pemanggilan token dapat dilakukan melalui struct token.

Berikut adalah enum dari Tokennya:

```
enum TokenType {
  // Literals & Identifiers
  intcon, realcon, charcon, string_tok, ident,

  // Operators
  notsy, plus_tok, minus_tok, times, idiv, rdiv, imod, andsy, orsy,
  eql, neq, gtr, geq, lss, leq, becomes,

  // Delimiters & Punctuation
  lparent, rparent, lbrack, rbrack, comma, semicolon, period, colon,

  // Keywords
  constsy, typesy, varsy, functionsy, proceduresy, arraysy, recordsy,
  programsy, beginsy, ifsy, casesy, repeatsy, whilesy, forsy, endsy,
  elsesy, untilsy, ofsy, dosy, tosy, downtosy, thensy,

  // Special
  comment, eof_tok, unknown_tok
}
```

Berikut adalah implementasi struct token:

```
struct Token {
```

```
  TokenType type;
  std::string lexeme;
};
```

2. File Lexer.hpp dan Lexer.cpp

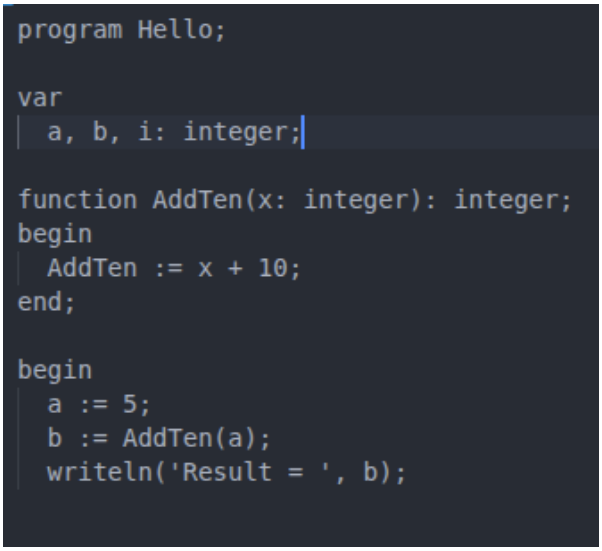
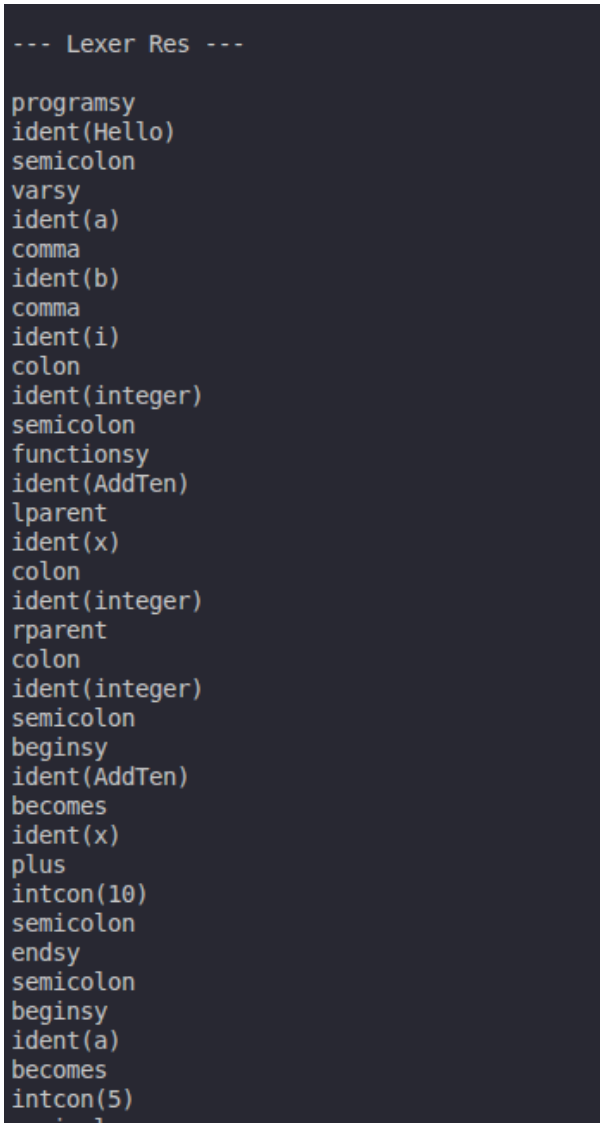
Kedua file ini bertugas sebagai lexical analyzer utama dalam program ini. Menggunakan pola arsitektur deklarasi dan implementasi, kedua file ini bekerja secara terintegrasi sebagai mesin pembaca kode sumber yang memvalidasi teks mentah dan mengkonversinya menjadi aliran token.

Didalam lexer.cpp terdapat switch pada methode nextToken() disinilah letak DFA nya diimplementasi. Dalam DFA formal, transisi ditentukan oleh State saat ini dan Karakter input. getNextToken() selalu memulai dari STATE_START (State 0), blok switch berfungsi sebagai pemilih jalur pertama berdasarkan karakter yang dibaca. Setiap case dalam switch mewakili sebuah busur (panah)/transisi DFA yang keluar dari STATE_START menuju state berikutnya. Di dalam case/transisi juga dapat divisualisasi via CLI namun tidak digunakan saat ini.

```
switch (current) {
    case ';':
        dfa.transition(';', STATE_SEMICOLON);
        return {semicolon, ";"};
    case '.':
        dfa.transition('.', STATE_PERIOD);
        return {period, "."};
    case ':':
        dfa.transition(':', STATE_COLON, "Goten: Colon");
        if (pos < source.length() && source[pos] == '=') {
            char next = source[pos++];
            dfa.transition(next, STATE_BECOMES, "Goten: Becomes");
            return {becomes, ":="};
        }
        return {colon, ":"};
    case '+':
        dfa.transition('+', STATE_PLUS);
        return {plus_tok, "+"};
   // Dan seterusnya …..
```

```
    default:
        dfa.transition(current, STATE_UNKNOWN);
        return {unknown_tok, std::string(1, current)};
}
```

BAB 3 : Pengujian

Input	Output
Case 1	
	
Case 2	

```
program LogicTest;
var
  x, y: integer;

  check: boolean;
begin
  x := 10;
  y := 20;

  if (x <> y) and (x <= y) then
  begin
    check := true;
  end
  else
  begin
    check := false;
  end;
end.
```

```
programsy
ident(LogicTest)
semicolon
varsy
ident(x)
comma
ident(y)
colon
ident(integer)
semicolon
ident(check)
colon
ident(boolean)
semicolon
beginsy
ident(x)
becomes
intcon(10)
semicolon
ident(y)
becomes
intcon(20)
semicolon
ifsy
lparent
ident(x)
neq
ident(y)
rparent
andsy
lparent
ident(x)
leq
ident(y)
rparent
thensy
beginsy
ident(check)
becomes
ident(true)
semicolon
endsy
elsesy
beginsy
```

Case 3

```
program CommentTest;
{ Ini adalah komentar satu baris }
(* Ini adalah komentar
   multi-baris Pascal klasik *)
begin
  writeln('Testing quotes: it''s working');
  { Komentar di akhir program }
end.
```

```
programsy
ident(CommentTest)
semicolon
comment({ Ini adalah komentar satu baris })
comment((* Ini adalah komentar
   multi-baris Pascal klasik *))
beginsy
ident(writeln)
lparent
string('Testing quotes: it')
string('s working')
rparent
semicolon
comment({ Komentar di akhir program })
```

Case 4

```
program ArrayTest;
const
  PI = 3.14;
var
  numbers: array[1..5] of real;
  i: integer;
begin
  for i := 5 downto 1 do
  begin
    numbers[i] := i * PI;
  end;
end.
program RecordTest;
type
  Point = record
    x, y: integer;
  end;
var
  p1: Point;
begin
  p1.x := 10;
  p1.y := 20;

  { Menguji apakah lexer bingung antara . (period) dan .. (range) }
  writeln('Coordinates: ', p1.x, ',', p1.y);
end.
```

```
programsy
ident(ArrayTest)
semicolon
constsy
ident(PI)
eql
realcon(3.14)
semicolon
varsy
ident(numbers)
colon
arraysy
lbrack
intcon(1)
period
period
intcon(5)
rbrack
ofsy
ident(real)
semicolon
ident(i)
colon
ident(integer)
semicolon
beginsy
forsy
ident(i)
becomes
intcon(5)
downtosy
intcon(1)
dosy
beginsy
ident(numbers)
lbrack
ident(i)
rbrack
becomes
ident(i)
times
ident(PI)
semicolon
endsy
semicolon
endsy
period
programsy
ident(RecordTest)
```

Case 5

```
program RecordTest;
type
  Point = record
    x, y: integer;
  end;
var
  p1: Point;
begin
  p1.x := 10;
  p1.y := 20;

  { Menguji apakah lexer bingung antara . (period) dan .. (range) }
  writeln('Coordinates: ', p1.x, ',', p1.y);
end.
```

```
programsy
ident(RecordTest)
semicolon
typesy
ident(Point)
eql
recordsy
ident(x)
comma
ident(y)
colon
ident(integer)
semicolon
endsy
semicolon
varsy
ident(p1)
colon
ident(Point)
semicolon
beginsy
ident(p1)
period
ident(x)
becomes
intcon(10)
semicolon
ident(p1)
period
ident(y)
becomes
intcon(20)
semicolon
comment({ Menguji apakah lexer bingung antara . (period) dan .. (rang
ident(writeln)
lparent
string('Coordinates: ')
comma
ident(p1)
period
ident(x)
comma
charcon(',')
```

BAB 4 : Kesimpulan dan Saran

4.1 Kesimpulan

Berdasarkan hasil perancangan dan implementasi *Lexical Analyzer* menggunakan pendekatan *Deterministic Finite Automaton* (DFA), dapat ditarik kesimpulan sebagai berikut:

1. Efektivitas DFA pada Lexer: Penggunaan DFA terbukti efektif dalam mengenali pola-pola bahasa reguler pada subset bahasa Pascal. Implementasi *Partial DFA* menggunakan struktur kontrol **switch-case** dan **while** di C++ memberikan performa eksekusi yang cepat dengan kompleksitas waktu $O(n)$ terhadap panjang input.
2. Determinisme Sistem: Program berhasil mempertahankan sifat deterministik. Penggunaan teknik *lookahead* pada operator multi-karakter (seperti `:=` dan `<=`) serta penanganan *dead state* pada blok **default** memastikan bahwa setiap karakter input hanya memiliki satu jalur transisi yang pasti tanpa adanya ambiguitas atau transisi ϵ.
3. Akurasi Tokenisasi: Integrasi antara **lexer.cpp** dan **keywordMap** berbasis *hash table* berhasil memisahkan antara *identifier* pengguna dan *reserved words* secara akurat. Penggunaan *safe casting* pada karakter input juga mencegah kegagalan sistem saat menghadapi karakter non-ASCII.
4. Halt Condition sebagai Batas Lexeme: Mekanisme *halt* pada *accepting state* berfungsi dengan benar sebagai penanda batas *lexeme* (Longest Match). Hal ini membuktikan bahwa DFA dalam *Lexer* adalah kumpulan mesin state kecil yang bekerja secara iteratif, bukan satu mesin tunggal yang memproses seluruh file sebagai satu string.

4.2 Saran

Untuk pengembangan lebih lanjut dari proyek *Lexical Analyzer* ini, disarankan beberapa peningkatan sebagai berikut:

1. Implementasi Error Recovery: Saat ini, *Lexer* hanya mendeteksi karakter ilegal sebagai **unknown_tok**. Pengembangan selanjutnya dapat menambahkan fitur *Error Recovery* (seperti *Panic Mode*) agar *Lexer* dapat melaporkan baris dan kolom lokasi kesalahan secara spesifik tanpa menghentikan proses *scanning* seluruh file.
2. Integrasi ke Syntax Analyzer (Parser): Proyek ini dapat dilanjutkan ke tahap *Syntax Analysis* dengan membangun *Parser* (misalnya menggunakan algoritma *Recursive Descent* atau *LL(1)*) untuk memvalidasi struktur kalimat berdasarkan aliran token yang dihasilkan.
3. Optimalisasi Table-Driven DFA: Jika jumlah *state* dan alfabet meningkat drastis, disarankan untuk mengonversi logika *switch-case* menjadi *Table-Driven DFA* (menggunakan matriks transisi 2D) untuk mempermudah pemeliharaan kode (*maintainability*).
4. Dukungan Karakter Unicode: Mengingat standar pemrograman modern, dukungan untuk enkoding UTF-8 dapat ditambahkan agar *Lexer* mampu memproses karakter internasional di dalam komentar atau *string literal*.

Lampiran

- Link Repository Github = https://github.com/kalkabena/XYZ-Tubes-IF2224-2026.git
- Link Workspace Diagram Transisi DFA = https://drive.google.com/file/d/1JneYAPqP7K_h_I6Zhn2QttBRz9nQYbru/view?usp=sharing
- Tabel Pembagian Tugas:

NIM	Nama	Pembagian Tugas	Persentase Kontribusi
13524114	Mirza Tsabita Wafa'ana	Membuat Laporan dan membuat diagram transisi DFA	25%
13524119	Nathanael Shane Bennet	Membuat source code program dan mengerjakan laporan	25%
13524130	Faris Wirakusuma Triawan	Membuat source code program dan mengerjakani Laporan	25%
13524144	Jonathan Harijadi	Mengerjakan Laporan dan membuat diagram transisi DFA	25%

Referensi

“Compiler Design - Lexical Analysis”
https://www.tutorialspoint.com/compiler_design/compiler_design_lexical_analysis.htm

“An Introduction to Makefiles”
https://www.gnu.org/software/make/manual/html_node/Introduction.html

Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). *Compilers: Principles, Techniques, and Tools (2nd Edition)*. Pearson Education. Halaman 109-150 mengenai *Lexical Analysis*.